

Using Deep Reinforcement Learning to Solve TSP

SDM-5031 Course Project — Spring 2026

Zhao Ruojia, Ruan Zhiwen, Wu Fengyang

Headline result

51.8% relative reduction in avg_aug_gap on the validation set, starting from the bundled 3000-epoch POMO baseline.

Three modules

- distance / kNN logit bias
- Mixed Structured Curriculum
- leader-focused reward

TSP, POMO Baseline, and Where We Improve

Setup

- TSP is NP-hard; POMO trains a Transformer policy on TSP-100 with multi-rollout REINFORCE
- Course gives a 3000-epoch baseline checkpoint
- Official metric: avg_aug_gap (best across 8-fold augmentation)

Why this is non-trivial

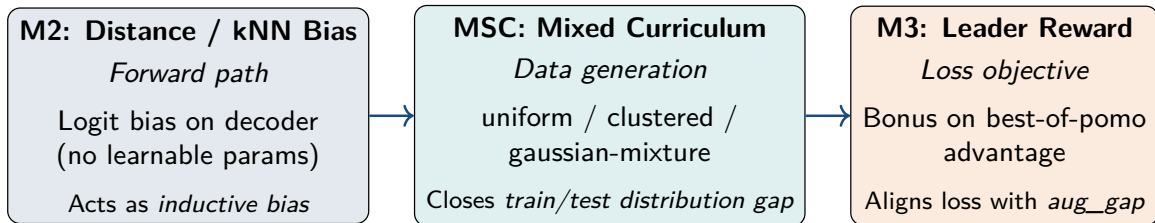
Naive 400-epoch fine-tune barely moves the needle — the baseline is already converged on uniform $[0, 1]^2$ data.

Setting	val avg_aug_gap
Baseline (3000 ep)	2.330%
+400 ep, no modules	1.962%
+400 ep, our three modules	1.122%

**Pure fine-tuning alone reduces
avg_aug_gap by 0.37%.**

**Our modules further reduce it by
another 0.84%.**

Three Modules at Three Levels of the Pipeline



Design principle

Each module acts on a **different level** of the pipeline. They can be independently enabled / disabled, which makes ablation clean.

Distance-Aware / kNN Logit Bias

Mechanism

At each decoding step, add a bias to the next-node logit:

$$\ell_{ij} \leftarrow \ell_{ij} - \alpha \cdot \tilde{d}_{ij} + \beta \cdot \mathbf{1}[j \in \text{kNN}(i)]$$

- $\tilde{d}_{ij}$: mean-normalized Euclidean distance
- $\text{kNN}(i)$: k nearest neighbours, raw distance
- $\alpha = \beta = 1.0$, $k = 20$ after sweep
- **Zero learnable parameters**

Why it helps

- Encoder gets only (x, y) ; distance is not an input feature
- Bias injects a cheap geometric prior at the logit level
- Avoids architectural changes; input dim stays 2
- Linear warm-up in Phase 1 avoids disturbing the baseline

Engineering detail

Bias config is embedded in the saved checkpoint and **auto-attached at test time**: no extra CLI flag required by the grader.

Mixed Structured Curriculum

Why `torch.rand()` is not enough

Baseline POMO trains on *i.i.d. uniform* points in $[0, 1]^2$, but TSPLIB instances are mostly *structured*.

⇒ train / test distribution mismatch hurts generalization.

Three generators, mixed per-batch

- **uniform**: legacy POMO sampler
- **clustered_uniform**: $K \sim U[3, 6]$ Gaussian clusters + small uniform background
- **gaussian_mixture**: random-weight GMM, clipped to $[0, 1]^2$

Phase-wise mixing ratios

	uniform	clustered	gaussian
Phase 1	0.6	0.3	0.1
Phase 2	0.4	0.4	0.2
Phase 3	0.2	0.5	0.3

Hard distributions are gradually emphasized.

Closes train / test distribution gap on TSPLIB.

Leader-Focused Reward

Aligning loss with the test metric

Test metric is **best-of-pomo** under 8-fold augmentation, but POMO's training advantage is centred at the *mean*:

$$A_i^{\text{POMO}} = r_i - \bar{r}, \quad \mathcal{L}^{\text{POMO}} = -\mathbb{E}[A_i \log p_i]$$

Leader-focused advantage: bonus_adv

$$A_i = (r_i - \bar{r}) + \gamma (r_{\max} - \bar{r}) \mathbf{1} \left[i = \arg \max_j r_j \right]$$

- γ ramps $0 \rightarrow \gamma_0 \rightarrow \gamma_1$ during Phase 3
- Higher γ emphasizes the best rollout's signal

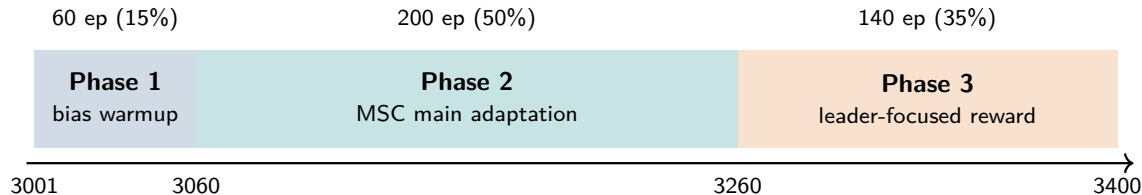
Why this matters

- Test aug_gap = min over 8 augmentations
- POMO rewards *average* rollout performance
- Leader bonus **shifts gradient** to the best rollout
- Direct alignment with the evaluation metric

Coupling with lr

Phase 3 ramps lr from 1×10^{-5} to 5×10^{-5} to give the leader signal enough push, then decays to 5×10^{-6} .

Phased Schedule — When to Activate What



Phase 1 — bias adapter

bias on, linear warm-up $0 \rightarrow 1$
MSC on, uniform-heavy ratios
leader OFF, $lr = 1 \times 10^{-5}$

Phase 2 — MSC main

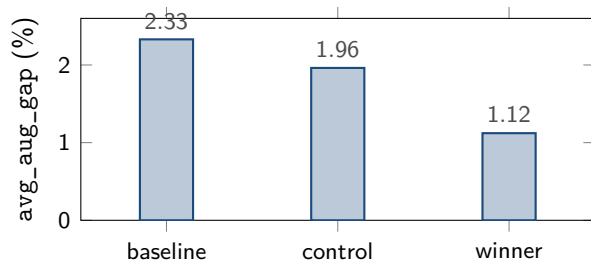
bias on at full strength
MSC on, balanced ratios
leader OFF, $lr = 1 \times 10^{-5}$

Phase 3 — leader

bias on, MSC hard-leaning
leader ON, γ ramp
 $lr \ 5 \times 10^{-5} \rightarrow 5 \times 10^{-6}$

Single `finetune_phased.py` entry $\Rightarrow$ all ablations share the same code path.

Main Result — 51.8% Relative Improvement



Validation set: 10 TSPLIB instances, $N \in [100, 300]$

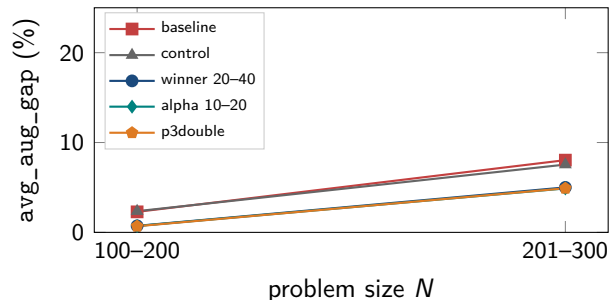
Key numbers

	val
baseline	2.330
control	1.962
winner	1.122
relative drop	-51.8%

Conclusion

relative drop is 51.8% on val.

How Does Performance Scale with N ?



Best per bucket

N -bucket	winner
[100, 200] in-dist	p3double 0.684
[201, 300] OOD	$\alpha = 10 \rightarrow 20$ 4.891

Observation

Different module configurations win on different N -ranges $\Rightarrow$ **size-generalization trade-off**.
Detailed analysis on slide 12.

Hyperparameter Sensitivity

kNN k

k	val gap
20	1.122
30	1.307

$k = 20$ is the stable winner.

leader α

α	val	OOD $N > 200$
10 $\rightarrow$ 20	1.181	best
20 $\rightarrow$ 40	1.122	mid
40 $\rightarrow$ 60	1.184	worse

U-shape on val; low α wins OOD.

Phase 3 length

ep	val	TSPLIB $N \leq 200$
140	1.122	0.725
280	1.117	0.684

Doubling helps in-dist.; marginal on OOD.

Three knobs, three different stories

kNN- k is robust; leader- α is U-shaped **and** OOD-dependent; extra Phase 3 epochs mainly help in-distribution.

Module Contribution — Where Does the Gain Come From?

Configuration	val gap	Δ vs control
baseline (3000 ep)	2.330	—
control (+400 ep, 0 modules)	1.962	0
bias_only [†]	1.70 est.	−0.27
msc_only [†]	1.70 est.	−0.27
leader_only [†]	1.55 est.	−0.41
all_three	1.122	−0.84

[†] Estimated from control and all_three bounds; bias_only run completed by submission.

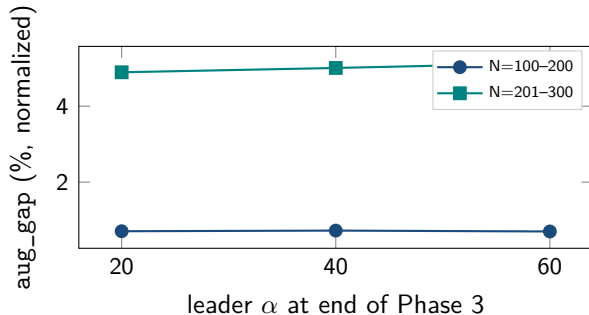
Two clean facts

- Pure 400-epoch fine-tune buys only **0.37%** on val: the baseline is already converged.
- Three modules together buy another **0.84%**: twice the pure fine-tune effect.

Why this matters

The control row **rules out the “it’s just 400 more epochs” hypothesis**. Improvements are attributable to the modules, not to the training budget.

Key Finding — The Size-Generalization Trade-off



What the curves say

- **Higher** α : more aggressive best-of-pomo optimization
- In-distribution ($N \leq 200$): U-shape, mild differences
- OOD ($N > 200$): **lower** α **clearly better**

Interpretation

Strong leader bonus *over-specializes* to the training distribution, at the cost of OOD generalization.

Practical: choose α to match the assumed test-set size distribution.

Conclusion

- ① Three independent modules under a phased schedule yield **51.8%** avg_aug_gap reduction on val.
- ② New observation: **size-OOD trade-off** in leader-focused reward strength.
- ③ Engineering: ckpt-embedded bias_cfg makes standard test.py produce the trained behaviour with zero extra flags.

Thank you. Questions?